

# Relatório Trabalho de Implementação 1 - Cifra de Vigenère

## (Segurança Computacional - 2026/2)

### Integrantes

- Alex Batista Resende (231025261)
- Luis Henrique Bessa de Melo (232009567)
- Marco Aurélio Gonçalves Fonseca (221021509)
- Thomas Jefferson (202033561)

### Abstract

Este relatório detalha a implementação e a criptoanálise estatística da cifra polialfabética de Vigenère através de um software desenvolvido em Python. O estudo demonstra, na prática, como a combinação do Índice de Coincidência com o teste Qui-Quadrado é capaz de quebrar o algoritmo e recuperar chaves desconhecidas em textos extensos. Os resultados reforçam a vulnerabilidade histórica de cifras clássicas e evidenciam a necessidade indispensável de protocolos criptográficos modernos para garantir a segurança em redes de computadores.

### Introdução:

A cifra de Vigenère, desenvolvida no século XVI, é um método clássico de criptografia por substituição polialfabética que utiliza uma palavra-chave para aplicar deslocamentos modulares variáveis sobre os caracteres de uma mensagem. Diferente das cifras monoalfabéticas simples, como a de César, esta técnica mascara os padrões diretos de frequência do idioma, exigindo metodologias criptoanalíticas mais avançadas para sua quebra, como o cálculo do Índice de Coincidência e a aplicação do teste de adequação estatística de Qui-Quadrado.

Este relatório descreve a implementação de um software completo em Python estruturado em duas etapas principais: o desenvolvimento de um cifrador/decifrador fiel à lógica matemática

original (preservando o formato original do texto) e a construção de um módulo de criptoanálise capaz de recuperar chaves desconhecidas explorando as fraquezas estatísticas inerentes ao tamanho finito da senha e à previsibilidade da linguagem natural. O documento está organizado em seções que abordam a fundamentação teórica da cifra e do ataque, a metodologia experimental e a análise dos resultados obtidos nos testes computacionais.

## Fundamentação teórica:

A cifra de Vigenère é um método de criptografia por substituição polialfabética que emprega uma palavra-chave para aplicar múltiplos deslocamentos sobre o texto claro. Matematicamente, a cifração ocorre pela soma do valor da letra original ( $P$ ) com a letra da chave ( $K$ ) em aritmética modular, dada pela equação  $C = (P + K) \bmod 26$ . A decifração reverte esse processo através da subtração modular, expressa por  $P = (C - K + 26) \bmod 26$ . Ao repetir a chave ciclicamente, a cifra mascara os padrões naturais de frequência do idioma.

Para quebrar a criptografia sem conhecer a senha, a análise de frequência simples é ineficaz, exigindo primeiro a descoberta do comprimento da chave. Esse passo é realizado utilizando o Índice de Coincidência (IC), que mede a probabilidade de duas letras sorteadas de um texto serem idênticas. A medida é obtida pela equação  $IC = \text{Soma}[f * (f - 1)] / [N * (N - 1)]$ , onde " $f$ " é a frequência absoluta de cada letra e " $N$ " é a quantidade total de caracteres. Idiomas naturais (como português e inglês) apresentam um IC em torno de 0.07, enquanto distribuições aleatórias marcam cerca de 0.038.

Na prática, o algoritmo fragmenta o criptograma em colunas baseadas em suposições de comprimento (ex: de 1 a 20). O tamanho de chave hipotético que gerar a média de IC mais próxima da linguagem natural indica a extensão exata da senha. Isso significa que as letras de uma mesma coluna sofreram um deslocamento único, reduzindo o problema a múltiplas cifras de César independentes.

Com o texto fatiado corretamente, aplica-se o teste estatístico Qui-Quadrado em cada coluna para determinar a letra específica da chave. A fórmula **Qui-Quadrado** =  $\text{Soma}[(O - E)^2 / E]$  calcula a distância entre as contagens de letras observadas ( $O$ ) e a distribuição esperada do idioma ( $E$ ). O algoritmo testa por força bruta os 26 deslocamentos possíveis em cada subconjunto; o deslocamento que resultar no menor valor estatístico revela o caractere correto da senha.

## Ambiente Experimental e Análise de Resultados

## Descrição do Cenário

O cenário experimental foi estruturado sobre um ambiente de desenvolvimento local, utilizando a linguagem de programação Python em sua versão 3. Esta linguagem foi escolhida por sua robustez no processamento de cadeias de caracteres e suporte a paradigmas funcionais. Uma vez que o escopo deste laboratório foca estritamente nos mecanismos de segurança e criptografia no nível de aplicação, não foi necessária a implementação ou simulação de uma topologia de rede física; todas as observações e processamentos ocorreram localmente no hardware do desenvolvedor.

As configurações do ambiente de software basearam-se no uso de bibliotecas nativas da linguagem para otimizar o roteiro do laboratório. Utilizou-se a biblioteca `collections` para a contagem de frequência de caracteres e a `unicodedata` para a normalização de strings. Além disso, o cenário de testes precisou ser previamente alimentado com configurações estáticas de parâmetros linguísticos, representadas pelos dicionários `FREQ_PT` e `FREQ_EN`. Estas tabelas contêm a frequência esperada de cada letra do alfabeto na língua portuguesa e inglesa, respectivamente, e foram fundamentais para calibrar o ambiente experimental e possibilitar o sucesso das métricas de criptoanálise discutidas na fundamentação teórica.

## Análise de Resultados

A análise prática foi dividida nas etapas lógicas implementadas no script, sendo cada resultado avaliado através das saídas geradas no terminal iterativo do sistema.

Na primeira atividade, referente à *Cifragem e Decifragem*, os testes validaram a implementação da cifra polialfabética de Vigenère. O resultado demonstrou que a função `vigenere_processar` operou de forma segura ao preservar a integridade estrutural da mensagem. A rotina `remover_acentos` aplicou a normalização NFKD, substituindo caracteres diacríticos pelas suas versões base (ex: "á" para "a"). Observou-se que caracteres não alfabéticos (como espaços e pontuações) foram corretamente ignorados no cálculo matemático de deslocamento ASCII. Um ajuste técnico crucial nesta fase foi garantir que o índice iterador da chave avançasse *apenas* ao encontrar letras; caso contrário, a presença de espaços na mensagem dessincronizaria a cifra com a chave, corrompendo a posterior decifragem.

Na segunda atividade, realizou-se a *Estimativa do Tamanho da Chave*. O algoritmo analisou o criptograma fatiando-o em diversas colunas, variando a hipótese de tamanho da chave ( $k$ ) de 1 até 20. Para cada tamanho, calculou-se a média do Índice de Coincidência (IC). Os resultados gerados na tabela de saída evidenciaram que textos cifrados por chaves polialfabéticas tendem a exibir um IC próximo à aleatoriedade (cerca de 0.038), enquanto os tamanhos corretos de chave geravam picos de IC próximos a 0.060 ou mais (comportamento de texto em linguagem

natural). Um problema encontrado nesta fase é que o algoritmo tendia a apontar múltiplos matemáticos da chave real como a resposta correta (por exemplo, sugerir tamanho 8 para uma chave de tamanho 4). A solução encontrada e implementada no código foi uma heurística de filtragem que busca o menor divisor comum entre os candidatos, priorizando aquele que ainda mantivesse um IC acima de 0.065.

Na terceira atividade, focada na *Descoberta da Chave*, o sistema separou o criptograma em subtextos, tratando cada um como uma Cifra de César simples. Para cada subtexto, testou-se todos os 26 deslocamentos possíveis do alfabeto. O resultado foi mensurado utilizando o teste estatístico do Qui-Quadrado ( $\chi^2$ ), que calculou a distância entre a frequência de letras obtida no deslocamento e a frequência ideal do idioma escolhido (FREQ\_PT ou FREQ\_EN). O algoritmo exibiu as três melhores pontuações de hipóteses de letras para cada posição, obtendo êxito total na reconstrução da chave na maioria dos cenários testados.

Por fim, o principal problema geral encontrado ocorreu durante a tentativa de quebrar criptogramas curtos (com menos de 100 caracteres). Devido à escassez de dados, a distribuição estatística do texto se afasta da média do idioma, fazendo o ataque do Qui-Quadrado e do IC falharem. A solução para esse problema foi implementar uma pausa na interface interativa da aplicação (input). Dessa forma, o programa sugere o tamanho mais provável da chave, mas permite que o usuário faça um ajuste manual e force um tamanho de chave específico baseado em erro e acerto, garantindo a flexibilidade da ferramenta.

## Conclusões

Em suma, o desenvolvimento prático deste projeto atendeu com excelência aos objetivos propostos na disciplina, implementando a cifra de Vigenère e atestando sua vulnerabilidade. Os resultados comprovam que a substituição polialfabética falha em ocultar padrões linguísticos quando a chave é reutilizada, permitindo que ferramentas estatísticas como o Índice de Coincidência e o Qui-Quadrado recuperem a mensagem original sem grande esforço computacional. Verificou-se, também, que a precisão desta criptoanálise é diretamente dependente do volume de texto interceptado. Conclui-se, portanto, que métodos criptográficos clássicos são insuficientes para garantir a confidencialidade, reiterando a necessidade de protocolos modernos no atual panorama das redes de computadores.

## Bibliografia

- Este projeto foi realizado de acordo com o roteiro do Projeto 1 da disciplina Segurança Computacional, ministrada pela professora Priscila Solis.

- Funcionamento da cifra de Vigenere: <https://www.youtube.com/watch?v=SkJcmCaHqS0>
- Quebrando a cifra de Vigenere: <https://www.youtube.com/watch?v=P4z3jAOzT9I&fbclid=IwAR0Aj7nSiyUmdf1 XaRI7>